

Table of Contents

1. Email Templates	1
1.1. What the framework supports	1
1.2. Architecture	1
1.3. Template sources and resolution order	2
1.4. Rendering templates	3
1.5. Tenant-scoped REST API	4
1.6. Onboarding workflow integration	5
1.7. Delivering email	6
1.8. When to use email templates	7
1.9. See also	7

1. Email Templates

Quantum provides a built-in email templating system with three important pieces:

- classpath defaults under `templates/email/{templateKey}/`
- realm-scoped overrides stored in MongoDB
- Qute-based rendering plus pluggable delivery

This makes email content part of the framework instead of forcing each product to hand-roll template loading, rendering, preview, and override behavior.

1.1. What the framework supports

Email templates are now first-class framework resources:

- `EmailTemplate` documents are stored in the `email_templates` collection.
- `DefaultEmailTemplateResolver` checks the current realm for an active override first, then falls back to the classpath.
- `EmailTemplateRenderService` renders `subject.txt`, `body.html`, and `body.txt` using Qute.
- `EmailTemplateResource` exposes tenant-scoped CRUD and preview APIs at `/settings/email-templates`.
- `TenantOnboardingWorkflow` now carries `defaultInviteEmailTemplateKey`, so onboarding/invite flows can refer to a named template instead of hard-coded content.

1.2. Architecture

The email system has three layers:

1. Template resolution
2. Rendering
3. Delivery

```
Application Code
|
v
EmailTemplateRenderService.render(realm, templateKey, context)
|
v
EmailTemplateResolver.resolve(realm, templateKey)
|
+---> Check DB for realm-specific override (active=true)
|           |
|           v (not found)
+---> Load from classpath: templates/email/{templateKey}/
|
v
```

```
EmailTemplateDefinition (unrendered Qute templates)
    |
    v
Qute Engine renders subject, HTML body, text body
    |
    v
RenderedTemplate --> build RenderedEmail --> EmailDeliveryService.deliver()
```

1.3. Template sources and resolution order

The `DefaultEmailTemplateResolver` follows this priority:

1. Realm database override
2. Classpath default
3. Empty result if neither source contains the template

That means products can ship sane defaults in source control, while each tenant can still customize messaging without forking code.

Classpath templates

Place template files on the classpath under `templates/email/{templateKey}/`:

```
src/main/resources/
  templates/
    email/
      welcome/
        subject.txt
        body.html
        body.txt
      password-reset/
        subject.txt
        body.html
        body.txt
```

Each file is optional except that rendering must ultimately produce:

- a non-blank subject
- at least one non-blank body variant (`html` or `text`)

Example:

subject.txt

```
Welcome {name}
```

body.html

```
<p>Hello {name}, welcome to realm <strong>{realm}</strong>.</p>
```

body.txt

```
Hello {name}, welcome to realm {realm}.
```

Realm overrides in MongoDB

Tenants can override any classpath template by storing an `EmailTemplate` document in the `email_templates` collection for their realm.

Field	Description
<code>refName</code>	Persistent reference for the template document. If omitted on create, the API derives it from <code>templateKey</code> by replacing <code>.</code> with <code>-</code> .
<code>templateKey</code>	Logical template id used by application code, such as <code>welcome</code> or <code>tenant-onboarding-invite</code>
<code>subjectTemplate</code>	Qute-formatted subject template
<code>htmlTemplate</code>	Qute-formatted HTML body
<code>textTemplate</code>	Qute-formatted text body
<code>active</code>	Enables or disables the override without deleting it
<code>sourceType</code>	Usually <code>REALM_OVERRIDE</code> ; can also be set to <code>CLASSPATH_DEFAULT</code> for metadata purposes
<code>sampleContextJson</code>	Optional example JSON payload showing the expected render context
<code>schemaVersion</code>	Optional version marker for template lifecycle tracking

1.4. Rendering templates

Inject `EmailTemplateRenderService` and call `render()`:

```
@Inject
EmailTemplateRenderService renderService;

RenderedTemplate rendered = renderService.render(
    realmId,
    "password-reset",
    Map.of(
        "name", user.getDisplayName(),
        "realm", realmId,
        "resetLink", resetUrl
    )
);
```

The render context can be:

- `Map<String, Object>`
- a Java bean
- a Java record
- another Jackson-convertible object

The original object is also exposed to Qute as `{context}`.

Validation rules

After rendering, the framework validates:

- subject must be non-blank
- at least one of HTML or text body must be non-blank

If either rule fails, rendering throws `IllegalStateException`.

1.5. Tenant-scoped REST API

The framework exposes authenticated tenant-scoped endpoints at `/settings/email-templates`:

Endpoint	Purpose
<code>GET /settings/email-templates</code>	List templates stored for the current realm
<code>GET /settings/email-templates/{refName}</code>	Fetch one template by refName
<code>POST /settings/email-templates</code>	Create a new realm-scoped template
<code>PUT /settings/email-templates/{refName}</code>	Update an existing realm-scoped template
<code>DELETE /settings/email-templates/{refName}</code>	Delete a realm-scoped template
<code>POST /settings/email-templates/preview</code>	Render either an existing template by <code>templateKey</code> or inline template content

Example create request:

```
{
  "templateKey": "tenant-onboarding-invite",
  "displayName": "Tenant onboarding invite",
  "description": "Invite email shown to tenant users during onboarding.",
  "active": true,
  "subjectTemplate": "You're invited to {tenantDisplayName}",
  "htmlTemplate": "<p>Hello {inviteeName}, use <a href='{inviteLink}'>this link</a> to get started.</p>",
}
```

```
"textTemplate": "Hello {inviteeName}, use {inviteLink} to get started.",
"sampleContextJson": "{\"inviteeName\":\"Sam\",\"tenantDisplayName\":\"Acme\",
\"inviteLink\":\"https://example.test/invite/abc\"}"
}
```

Example preview request against a stored template:

```
{
  "templateKey": "tenant-onboarding-invite",
  "context": {
    "inviteeName": "Sam",
    "tenantDisplayName": "Acme Logistics",
    "inviteLink": "https://app.example.com/invite/abc123"
  }
}
```

Example preview request using inline content instead of a stored template:

```
{
  "templateKey": "preview-only",
  "subjectTemplate": "Welcome {firstName}",
  "textTemplate": "Hello {firstName}",
  "context": {
    "firstName": "Sam"
  }
}
```

1.6. Onboarding workflow integration

`TenantOnboardingWorkflow` now includes `defaultInviteEmailTemplateKey`.

This is the template key the tenant onboarding flow should use when it needs to render an invite email. The framework default key is:

```
tenant-onboarding-invite
```

That value is configuration, not a guaranteed built-in classpath template. In practice:

- your application can ship a classpath template with that key
- a tenant can override that key in the database
- the workflow can be updated to point at a different key per realm

Representative workflow update payload:

```
{
```

```

"displayName": "Tenant User Onboarding",
"inviteRequired": true,
"registrationRequired": true,
"defaultInviteEmailTemplateKey": "tenant-onboarding-invite"
}

```

This separation is useful because the onboarding workflow owns *which template key to use*, while the email-template subsystem owns *how that template is resolved and rendered*.

1.7. Delivering email

Build a `RenderedEmail` from the rendered output and pass it to an `EmailDeliveryService`:

```

@Inject
EmailDeliveryService deliveryService;

RenderedEmail email = RenderedEmail.builder()
    .from("noreply@example.com")
    .to(List.of(user.getEmail()))
    .subject(rendered.subject())
    .htmlBody(rendered.htmlBody())
    .textBody(rendered.textBody())
    .messageStream("outbound")
    .build();

deliveryService.deliver(email);

```

Postmark integration

The framework ships with `PostmarkEmailSender`.

```

postmark.api-key=${POSTMARK_API_KEY}
postmark.default-from-email-address=noreply@yourdomain.com

```

If either property is missing, the sender logs and returns without sending. That keeps local development and tests safe when no real delivery credentials are configured.

Custom delivery services

You can replace delivery with your own implementation:

```

@ApplicationScoped
@Alternative
@Priority(1)
public class SesEmailSender implements EmailDeliveryService {
    @Override
    public void deliver(RenderedEmail email) {

```

```
        // send via AWS SES, SendGrid, etc.  
    }  
}
```

Custom template resolvers

You can also replace template resolution entirely:

```
@ApplicationScoped  
@Alternative  
@Priority(1)  
public class CmsBackedTemplateResolver implements EmailTemplateResolver {  
    @Override  
    public Optional<EmailTemplateDefinition> resolve(String realm, String templateKey)  
    {  
        // load from S3, CMS, Git, etc.  
    }  
}
```

1.8. When to use email templates

Use the framework email-template subsystem when:

- the output is user-facing email content
- tenants may need per-realm content overrides
- you want previewable Qute templates instead of hard-coded strings

Do not use email templates for:

- provisioning reference data, which belongs in [seed packs](#)
- multi-step operational setup, which belongs in [BootstrapPacks](#)

1.9. See also

- [Tenant onboarding and access invites](#)
- [Seed packs and declarative tenant seeding](#)
- [Bootstrap Packs](#)
- [Configuration reference](#)